Final Project: Anonymous Message Board

AI Disclosure: Google AI and Copilot were used, along with some coding assistance from a colleague at AWS. All were used exclusively to streamline the html and python code and work towards ensuring all languages and APIs connect properly. More on this in the detailed write-up.

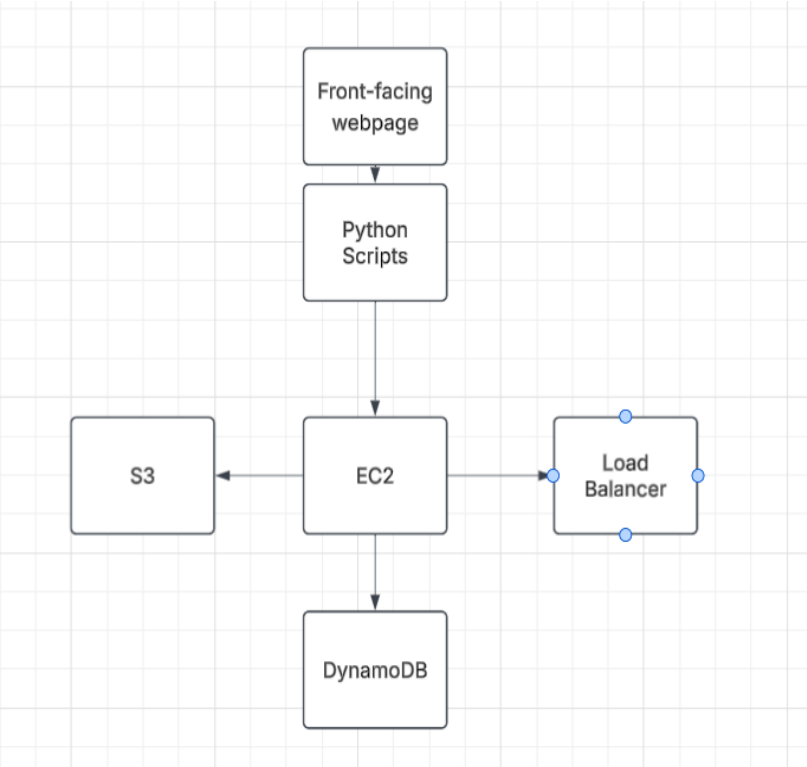
**Executive Summary:**

As a millennial, I grew up a lot with BBS, forums, and even 4chan. And while I enjoy Twitter (for the most part), its not quite the same.
This project was almost a blog instead, but LiveJournal and WordPress fill that gap so thoroughly that it felt like a waste.
As such, it became apparent that this needed the following features: EC2 for computing; DynamoDB support to maintain any new posts; S3 for file hosting (and backup/redundancies); and finally, a load balancer to ensure it stays up as much as possible.



Sometimes, you just want to scream into the void.
(This is the banner image for the site.)

**Architecture:**




1 - User sees html page, can fill out form and submit post, running on EC2.

2 - Python scripts act as a bridge between EC2 and html page to the DynamoDB.

3 – S3 Hosts original files, such as HTML, python, and the banner image for the webpage.

4 – Load balancer to ensure page can always be available, and has resources necessary for any future EC2 instance.

EC2 was chosen for a stable compute engine. To have an interactive webpage that constantly updates and hosts new information, S3 alone just wouldn't cut it. Going Serverless via Lambda or Fargate may have worked, but would have dramatically increased complexity and testing time. Bluntly, EC2 was just more comfortable and faster.

Deployment of infrastructure is handled via CloudFormation. While my yaml (clearly) isn't perfect (and has many things commented out to streamline other requirements) I still found it more effective for my needs than even a Golden Image or any other sort of backup. As above, testing is also streamlined as one or two tweaks in code and a Stack Change Set actually wound up being less difficult (especially after Mod 8 Lab 1).

S3 is used to host both the banner image file displayed on the site (making it rapidly changeable, either via S3 or the EC2 instance as necessary) to maintain backups and default state. Here is where the html and python files are also stored. Mostly a matter of convenience and quick referencing from any location.

As a small addendum, I used html and python due to my overall familiarity with them, but I wonder if using javascript instead may have mitigated some of my troubles throughout development.

Perhaps the most relevant piece of the architecture is the DynamoDB implementation. The Python is used to communicate to this, and is intended to create a consistent, persistent record of any and all posts made to the site, with "moderator" access only being provided to the developer/IAM users. Unironically, I do not even know what else I could have done for this system otherwise that wouldn't have come with dozens of other issues (even if I got S3 to handle this, the costs could easily get astronomical overnight). This was, what felt to me, the only logical answer.

Lastly, a load balancer. This is here for only one reason (other than assignment requirements), which is to try to keep the page up longer, easier. Pages that are similar to this (such as bbs/imageboards) tend to get a lot of traffic, and just one EC2 instance would not be enough for any more than a few people to use this.

**Implementation:**

In order to get this all working, the first step I took was to use the yaml file from Module 8 lab 1, and modifying the heck out of it to support other launch patterns. I worked a bit with html and some google AI answers to get everything rolling, only to run into problem after problem after problem in the Python implementation (couldn't get the variables right, all the ways code can communicate with each other).

I literally called on a friend / colleague who works for AWS for tips and pointers. We sat down together to look over my code, hammer it out, used his work Copilot to smooth out edges, and after several days of myself (and several hours with my colleague) bashing our heads into the wall, I finally found that boto3 never installed properly.
I manually installed them via my ec2 instance, and suddenly everything worked. I felt a mix up relief and newfound frustration that something so simple caused days of anguish – but lesson learned for next time.

Beyond that, the yaml file and stack handle *almost* everything. Once the files are in an S3 bucket, use the EC2 console to put everything where it needs to go (/var/www/html for index.html and banner.jpg, /var/www/cgi-bin for python files), and everything should just run.

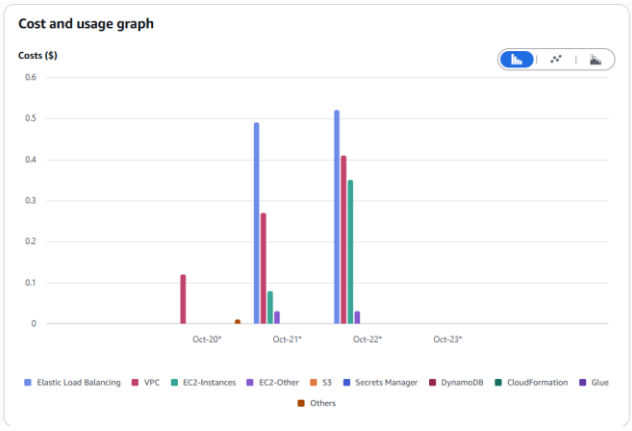
More to follow, however in the final section of this paper.

**Architectural concerns:**

Security and reliability are actually a bit of a concern.

I don't believe the Load Balancer is *quite* enough, and given that the EC2 terminal commands aren't *all* included in the yaml (or any other script) means that any EC2 failure…will not lead to an automatic recovery.

Security could be improved, largely through use of https. IAM role is used to perform all tasks along with security group. As well, http access from anywhere is currently allowed, which would create some vulnerabilities.




In terms of cost, at the moment this service appears to cost approximate $1.00 per day (which is honestly horrendous). Most of the cost comes from load balancing, followed by VPC, and then EC2 instances. Removing the load balancer and deliberately planning for more EC2 via provisioning would save a lot in the long run (if I intended to keep this running). Alternatively, updating and altering the architecture to find a simpler solution would probably pay dividends; I feel like this project could be done in a significantly simpler fashion that would be much more cost effective.

**Well-Architected self-review:**

1 – Operational Excellence: It does what it says on the tin. All code is stable, it runs, and any effort needed to change or update anything requires minimal hassle in the grand scheme of things. However, I am concerned with the potential for bloat, especially with old leftover commented-out errors or deprecated functionality that I left in mostly to give myself a paper trail and for any testing purposes.

2 – Reliability: If left to its own devices and used as intended, it runs great with no issues, and runs quickly and smoothly (including error messages if something is incorrect). However, I have deep concerns about load balancing and recovery from a catastrophe; I could easily rebuild it, and the DynamoDB is there to persist all posts (which is *great*), but if the EC2 were to go down, I would have to manually restore it, rather than it being automatic.

3 – Cost Optimization: Not great. Some of that stems from testing, some of it stems from assignment requirements, but there is no reasonable excuse for something as simple as this should cost $1.00 / day average. Fortunately, the S3 serves *primarily* as a storage for

files when EC2 spins up, so the data is rarely accessed, saving a small amount of money here and there.

**What I would do differently:**

God, where do I begin?

The idea itself is not stellar. There are just some things about the older internet I miss and would love back. But that's less relevant here.

I would evaluate if any of the infrastructure can use Javascript instead of Python, and if it would be more efficient. I would fix all the files until the project is able to be fully deployed in a single step. I would make the html page better. I would clean up all the code.

More than anything, I just wish I had more time. Not that the 3 weeks from the initial pitch wasn't enough time; but the fact that many of the things I wanted to implement, I didn't understand until the week of the assignment, while also maintaining that week's homework (labs) and trying to reconcile the rest of my classes. With more time to develop cleaner code, and have a better understanding of how each system communicates with one another, anything I build in the future will be a heck of a lot better than this.